Ejercicio 5. Leaflet



Contenido

Creación de un mapa sencillo con la API de Leaflet 3
Primeros pasos 3
Estilo del mapa 4
Código de inicialización del mapa 4
Capas teseladas WMS 8
Añadiendo capas de interfaz de usuario al mapa 10
Insertar un marcador 10
Cómo insertar un icono 11
Capas vectoriales 12
Agregar un círculo 12
Agregar polígonos 13
Agregar etiquetas 13
Crear una línea 14
Trabajando con grupos de capas 14
Trabajando con eventos 16
Controles 16
Control de escala 17
Control de capas 18
Control de títulos y subtítulos 22
Control de leyenda. Añadiendo una bonita leyenda al mapa 24
Capas vectoriales. El formato GeoJSON 26
Trabajando con archivos GeoJSON en Leaflet 27
Un poco de color 31
Interactuando con el mapa 34
Añadir una capa WFS de GeoServer y mostrar en una ventana emergente sus atributos mediante AJAX 35
Extendiendo la funcionalidad de Leaflet mediante plugins 38
Insertando un control de búsqueda de atributos en una capa estática GeoJSON 38

Creación de un mapa sencillo con la API de Leaflet

Como hemos hecho anteriormente, el primer paso para la creación de cualquier mapa web JavaScript es elaborar el marco HTML básico. Esta es la estructura que tendrá nuestro HTML:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0,
maximum-scale=1.0, user-scalable=no" />
        <style>
        </style>
</head>
        <body>
                <div id = 'map'>
                </div>
        </body>
</html>
```

Las etiquetas <meta> que hemos definido sirven para definir la codificación de caracteres y la visualización en pantalla. Esta última combinada con los estilos que veremos más adelante hacen que el mapa ocupe toda la pantalla del dispositivo independientemente de su tamaño. Es decir que el mapa puede verse en un dispositivo móvil o en la pantalla del ordenador.

Utiliza un editor de texto básico. Copia el texto anterior en un archivo nuevo y guardarlo en la ruta: `\data_dir\www` con el nombre `L_map1.html`.

Primeros pasos

En primer lugar incluiremos en la cabecera <head> de nuestra página web la librería JavaScript `leaflet.js` y la hoja de estilo `leaflet.css` :

```
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
```

Dentro de la etiqueta `<body>` encontramos el marcado para el mapa, que genera un único elemento de documento. También damos al contenedor un atributo id para que podamos hacer referencia a él en nuestro código:

```
<div id= 'map'></div>
```

Estilo del mapa

Nos aseguramos que el contenedor de mapa tenga un tamaño definido, dentro de la etiqueta `<style>` lo fijamos en CSS:

```
        body {
                padding: 0;
                margin: 0;
                }
        html, body, #map {
                height: 100%;
                }
```

Código de inicialización del mapa

Crearemos un mapa del centro de Valladolid con la capa base de OpenStreetMap. Primero inicializaremos el mapa y fijaremos su vista a nuestras coordenadas geográficas y un nivel de zoom. Vamos a añadir el siguiente código dentro del `<script>`:

```
var map = L.map('map', {
    center: [41.66, -4.72],
    zoom: 15,
});
```

`L.map` es la clase central de la API. Se usa para crear y manipular el mapa.

La sintaxis de la clase **`L.map`** es la siguiente:

```
L.map( <HTMLElement|String> id,

<Map options> options? )
```

A continuación, añadimos un mapa base como **`tile layer`**, en este caso las imágenes de OSM. Crear un **`tile layer`** lleva implícito establecer la URL, el texto con la atribución y el máximo nivel de zoom de la capa:

```
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; <a href="http://osm.org/copyright">OpenStreetMap</a>'
+ 'contributors',
maxZoom: 18
}).addTo(map);
```

Para incluir esta capa en nuestro mapa como opción, vamos a crear una variable llamada OSM y la añadimos como *layer* a la clase **map**.

```
var OSM = L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; <a href="http://osm.org/copyright">OpenStreetMap</a>'
+ 'contributors',
        maxZoom: 18
}); // eliminado el .addTo(map) porque se añade desde la clase map
```

Y modificamos la creación del mapa para añadir la opción de incluir las capas:

```
var map = L.map('map', {
    center: [41.66, -4.72],
    layers: OSM,
    zoom: 15,
});
```

Recuerda que es necesario crear la variable antes de poder utilizarla en el mapa.

Por defecto, en Leaflet la rueda del ratón (el *scroll*) sirve para acercarse o alejarse en el mapa. Sin embargo, esta acción no es *responsive* si trabajamos en entornos móviles, por lo tanto conviene modificar la opción **scrollWheelZoom** a falso:

```
var map = L.map('map', {
    center: [41.66, -4.72],
    layers: OSM,
    zoom: 15,
    scrollWheelZoom: false,
});
```

Por último, añadiremos un control de escala al mapa:

```
L.control.scale()

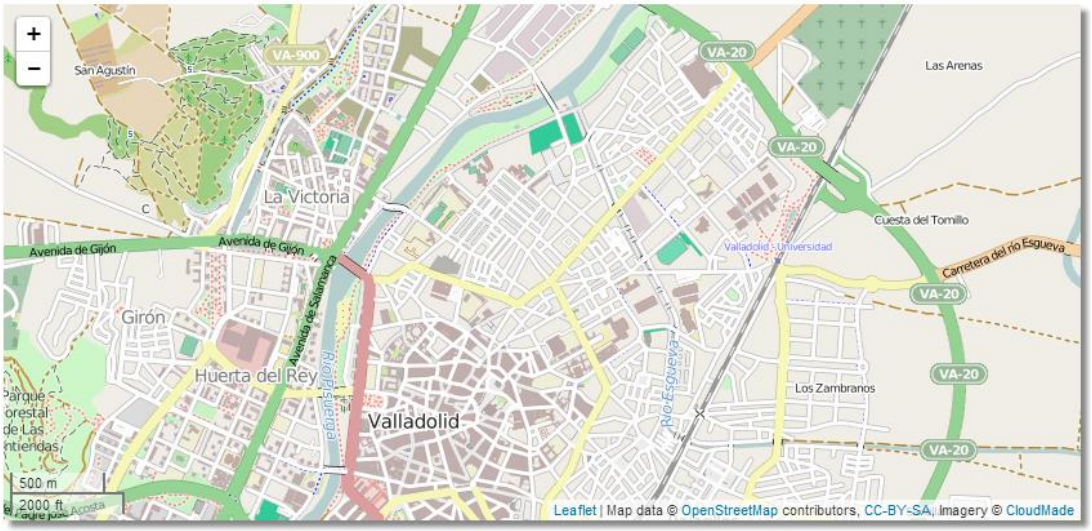
.addTo(map);
```



Importante: Para evita errores, no debemos abrir los visores web haciendo doble clic sobre ellos. Debemos abrirlos escribiendo la siguiente url en el navegador web:

http://localhost:8080/geoserver/www/L_map1.html

El resultado sería un mapa como este:



El código fuente utilizado es el siguiente:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-
scale=1.0, user-scalable=no" />
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

	<style>
		body {
		padding: 0;
		margin: 0;
		}
		html, body, #map {
		height: 100%;
		}

</style>
</head>
<body>
<div id="map"></div>
<script>
var OSM = L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
attribution: '&copy; <a href="http://osm.org/copyright">OpenStreetMap</a>'
		+'contributors',
		maxZoom: 18
	});

	var map = L.map('map', {
		//scrollWheelZoom: false,
		center: [41.66, -4.72],
		zoom: 13,
		layers: OSM
	});

	L.control.scale().addTo(map);
</script>


</body>
</html>
```

Ejercicio nº 1. Pega el resultado de tu primer mapa realizado con Leaflet (L_map1.html) en la plantilla de resultados.

Capas teseladas WMS

Ahora que ya tenemos una capa base, vamos a añadir nuestra capa de ciudades como un WMS, a través de la clase **`L.tileLayer.WMS`**

Haz una copia del archivo `L_map1.html` y renómbralo como `L_map2.html`

En primer lugar, vamos a cambiar el nivel de zoom a 4.

```
zoom: 4,
```

Ahora añadiremos debajo de la capa de OpenStreetMap el código necesario para agregar una capa WMS:

```
var cities = L.tileLayer.wms("http://localhost:8080/geoserver/wms", {
    layers: 'cities',
    format: 'image/png',
    transparent: true,
    tiled: true,
    attribution: "Natural Earth"
});
```

Para forzar a que se utilicen las capas teseladas hemos añadido el parámetro: `tiled: true`.

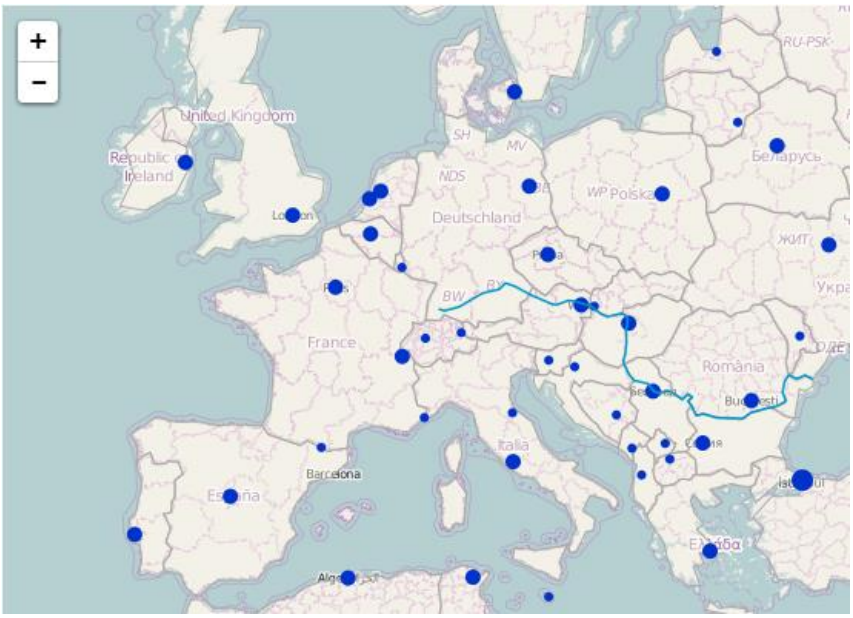
Ahora agregamos la capa a nuestro mapa:

```
layers: [osm, cities]
```

Ejercicio nº2. Añade al mapa las capas *rivers, countries* y *cities*.

En la capa cities utiliza la propiedad `styles` para aplicarle el estilo **cities_filter.** Haz una captura de pantalla con el resultado.

El orden en el que se localicen en el código determinará el orden en el que se van representando las capas. Por lo tanto, deberás jugar con el orden de las capas y con la transparencia de las capas.


+
−
United Kingdom
Republic of Ireland
Deutschland
France
España
Barcelona
Italia
Беларусь
Україна
România
Bucuresti
WP Polska

Añadiendo capas de interfaz de usuario al mapa

Insertar un marcador

Ahora vamos a agregar fácilmente otros elementos al mapa. Haz una copia del archivo `L_map1.html` y renómbralo como `L_map3.html`

Primero creamos un objeto marker utilizando la clase `L.marker:`

```
var marker = L.marker(
       [41.66, -4.71]
).addTo(map)
```

Debemos incluir esta sentencia debajo de la creación del objeto mapa, ya que estamos utilizando el método .`addTo`

Podemos añadir algunas opciones más a este marcador:

`draggable` indica si el marcador puede ser movible o no con el ratón.

`title`: Texto para la información sobre la herramienta que aparece sobre el indicador.

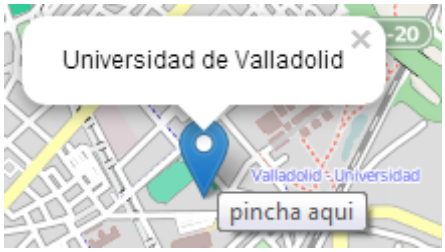
Podemos añadir una ventana emergente con un texto que se abrirá al hacer clic sobre el marcador. Para ello utilizamos el método `bindPopup:`

```
.bindPopup('Universidad de Valladolid');
```

Si además queremos que el mapa solo muestre un pop up a la vez podemos utilizar el método `.openPopup()`

Veamos cómo quedaría el código del marcador:

```
var marker = L.marker([41.66, -4.71], {
       draggable: true,
       title: 'pincha aquí'
}).addTo(map)
    .bindPopup('Universidad de Valladolid')
    .openPopup();
```



Este marcador podemos personalizarlo incluyendo algún icono que tengamos.

Cómo insertar un icono

Un icono marcador en Leaflet se define por un objeto `L.Icon`, que es pasado como una opción cuando creamos marcadores. Vamos a crear un icono con el logo de MappingGIS:

```
//Añadir un icono de marcador
var myIcon = L.icon({
    iconUrl: 'https://openlayers.org/assets/theme/img/logo70.png',
    iconSize: [30, 30],
    popupAnchor: [-1, -10]
});
```

Ahora ponemos el marcador en el mapa:

```
var marker2 = L.marker([41.65246, -4.6982], {icon: myIcon})
.addTo(map)
.bindPopup("Localización de <b>MappingGIS.com </b>");
```



Capas vectoriales

Agregar un círculo

Para añadir un círculo sobre el mapa utilizaremos la clase `L.circle`. Debemos especificar las coordenadas del centro como primer argumento (`[41.658, -4.725]`); el radio en metros como segundo argumento (`700`); y podemos controlar qué aspecto tendrá incluyendo opciones como último argumento (¡opcional y dentro de corchetes!) cuando creamos el objeto:

```
//Añadir un círculo
var circle = L.circle([41.658, -4.725], 700, {
    color: 'red',
    fillColor: '#f03',
    fillOpacity: 0.5
}).addTo(map);
```

Añadimos un *popup* a nuestro círculo:

```
circle.bindPopup("Centro de la ciudad");
```



Agregar polígonos

Para añadir un polígono sobre el mapa utilizaremos la clase `L.polygon`. Debemos especificar las coordenadas de los vértices:

```
//Añadir un polígono
var polygon = L.polygon([
        [41.67778, -4.72163],
        [41.67349, -4.71511],
        [41.66958, -4.72017],
        [41.67169, -4.72515]
]).addTo(map)
.bindPopup ('Mercaolid');
```



Agregar etiquetas

Aprovechando esta capa vectorial, a la que hemos denominado `polygon` vamos a añadirle una etiqueta que muestre un texto.

Para ello usamos el método `bindTooltip` de la clase `L.Tooltip`.

```
polygon.bindTooltip ("Soy una etiqueta", {
        permanent: false
})
```

Como hemos visto en la teoría, la clase `tooltip` tiene varias opciones: `pane, offset, direction, permanent, stiky, opacity.`

En el ejemplo anterior hemos añadido dentro de los corchetes la opción `permanent`, pero como ves su valor está establecido a `false`, que es el valor que adquiere por defecto. Modifica este valor a verdadero (`true`) y añade alguna opción más.

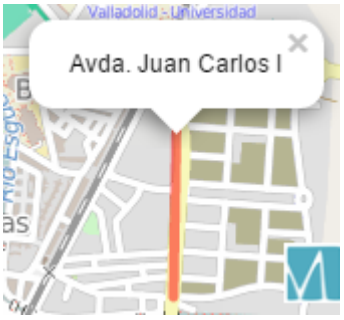
Crear una línea

Para añadir una línea sobre el mapa utilizaremos la clase `L.polyline`. Debemos especificar las coordenadas de los vértices:

```
//Añadir una línea

var pointA = new L.LatLng(41.65925, -4.70404);
var pointB = new L.LatLng(41.65166, -4.70421);
var latlngs = [pointA, pointB];

var polyline = L.polyline(
latlngs, {color: 'red'}
)
.addTo(map)
.bindPopup ('Avda. Juan Carlos I');
```



Ejercicio nº3. A las geometrías vistas añade además un marcador circular denominado circleMarker, debes utilizar la clase `L.circleMarker`.

Haz una captura de pantalla con el resultado del mapa `L_map3.html`

Trabajando con grupos de capas

En este ejercicio crearemos un grupo de capas que contenga todas las geometrías creadas hasta ahora, para gestionarlas de formas más sencilla.

Haz una copia del archivo `L_map3.html` y renómbralo como `L_map4.html`

Comenta TODOS los métodos `.addTo(map)` de las geometrías que participarán en el grupo, que serán:

- polyline,
- polygon,
- circle,
- circleMarker,
- marker,
- marker2

Deberían quedar de este modo

```
var circle = L.circle([41.658, -4.725], 700, {
    color: 'red',
    fillColor: '#f03',
    fillOpacity: 0.5
})//.addTo(map)
;
```

Abre el mapa. No deberías ver ninguna geometría.

Ahora vamos a crear un grupo de capas que incluya todas las geometrías y las añadimos al mapa:

```
var geometrias = L.layerGroup([polyline, polygon, circle, circleMarker, marker,
marker2])
        .addTo(map);
```

Utilizando el método `eachLayer` iteramos sobre cada una de las capas del grupo para mostrar un mensaje en una ventana emergente:

```
geometrias.eachLayer(function (layer) {
        layer.bindPopup('Pertenezco a un grupo de geometrías');
});
```

Ejercicio nº4. Haz una captura de pantalla con el mapa L_map4.html y el mensaje al hacer clic sobre un elemento.

Trabajando con eventos

Cada vez que algo ocurre en Leaflet, por ejemplo, cuando el usuario hace clic en un marcador o el mapa cambia de zoom, **el objeto** correspondiente envía un **evento** que se puede suscribir a una función. Esto nos permite reaccionar a la interacción del usuario.

Haz una copia del archivo `L_map4.html` y renómbralo como `L_map5.html`

```
function onMapClick(e) {
	alert("Has hecho clic en " + e.latlng);
	}

map.on('click', onMapClick);
```

Cada objeto tiene su propio conjunto de eventos. El primer argumento de la función es el objeto del evento, que contiene información útil sobre el evento que ocurrirá.

En este caso el objeto del evento click (`e` en nuestro ejemplo) tiene la propiedad `latlng`, que es la ubicación en la que ocurre el clic.

Ejercicio nº 5. Haz una captura de pantalla con el resultado del mapa `L_map5.html`

Controles

Leaflet por defecto solamente incluye dos controles por defecto: zoom y atribución.

La clase base para todos los controles es **L.control**. Implementa la interfaz IControl.

La sintaxis es:

```
L.control( <Control options> options? )
```

Las opciones irán siempre entre corchetes {}.

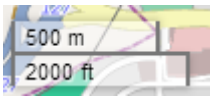
Una opción común para todos los controles es **position**: la posición inicial del control en el mapa.

Las posiciones posibles son: `topleft, topright, bottomleft, bottomright`.

Control de escala

Ya hemos visto en el ejercicio nº1 cómo incluir un control de escala:

```
L.control.scale()
.addTo(map);
```



Haz una copia del archivo `L_map2.html` y renómbralo como `L_map6.html`

Vamos a incluir ahora algunas opciones. En la referencia de la API (http://leafletjs.com/reference.html#control-scale) podemos ver que la posición por defecto del control es `bottomleft`.

Options

Option	Type	Default	Description
position	String	'bottomleft'	The position of the control (one of the map corners). See control positions.
maxWidth	Number	100	Maximum width of the control in pixels. The width is set dynamically to show round values (e.g. 100, 200, 500).
metric	Boolean	true	Whether to show the metric scale line (m/km).
imperial	Boolean	true	Whether to show the imperial scale line (mi/ft).
updateWhenIdle	Boolean	false	If true, the control is updated on moveend, otherwise it's always up-to-date (updated on move).

Vamos a incluir una opción para modificar la posición del control a la parte inferior derecha:

```
L.control.scale({position: 'bottomright'})
.addTo(map);
```

Las opciones `metric` e `imperial` vienen habilitadas por defecto (valor `true`), pero se pueden cambiar a `false`:

```
L.control.scale({imperial: false}).addTo(map);
```

Por lo tanto, al incluir varias opciones el resultado sería:

```
L.control.scale({
    position: 'bottomright',
    imperial: false
}).addTo(map);
```

Ejercicio nº6. Cambia el valor del máximo ancho del control de escala a 75 pixeles. Haz una captura de pantalla con el resultado del mapa `L_map6.html`

Control de capas

Leaflet tiene un bonito control que nos permite controlar que capas queremos ver en el mapa.

Hay dos tipos de capas:

- capas base (*baseLayers*) que son mutuamente excluyentes (sólo una puede ser visible en el mapa).
- Capas superpuestas (*overlays)*: el resto de capas que ponemos sobre las capas base.

Haz una copia del archivo `L_map2.html` y renómbralo como `L_map7.html`

En este ejercicio, queremos tener una capa base y dos *overlays* que se puedan activar y desactivar. Vamos a crear primero las variables para las capas:

```
var osm = new L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; <a href="http://osm.org/copyright">OpenStreetMap</a>
contributors',
        maxZoom: 18
});
```

```
var paises = new L.tileLayer.wms("http://localhost:8080/geoserver/wms", {
    layers: 'countries',
    format: 'image/png',
    transparent: true,
    attribution: "NaturalEarth"
});
var ciudades = new L.tileLayer.wms("http://localhost:8080/geoserver/wms", {
    layers: 'cities',
    format: 'image/png',
    transparent: true,
    attribution: "NaturalEarth"
});
```

A continuación, crearemos dos objetos. Uno contendrá nuestra capa base y el otro contendrá nuestras capas *overlays*. Son objetos simples con pares de clave/valor.

La **clave** es la que establece el **texto** en el control (por ejemplo, "Ciudades"). El valor correspondiente es la referencia a la capa (por ejemplo, `ciudades`).

```
var baseMaps = {
    "OpenStreetMap": osm
};

var overlayMaps = {
    "Paises": paises,
    "Ciudades": ciudades
};
```

El propio control ya añadirá las capas de la leyenda al mapa, por lo que no sería necesario añadir cada capa al mapa. Pero es necesario indicar que queremos que estas capas se añadan al iniciar el mapa:

```
var map = L.map('map', {
    center: new L.LatLng(41.66, -4.72),
    zoom: 4,
    layers: [osm, paises, ciudades]
});
```

Ahora lo que queda es crear el control de capas y añadirlo al mapa. El primer argumento que tenemos que pasar al crear el control de capas es el objeto de las capas base. El segundo argumento es el objeto de las capas superpuestas.

```
L.control.layers(baseMaps, overlayMaps).addTo(map);
```

Ambos argumentos son opcionales. Por ejemplo, se puede incluir el objeto capas de base omitiendo el segundo argumento:

```
L.control.layers(baseMaps).addTo(map);
```

O solo un objeto de las capas superpuestas, omitiendo el primer argumento introduciendo un **null**

```
L.control.layers(null, overlayMaps).addTo(map);
```

De igual forma si solo queremos introducer capas base, pero sí incluir opciones, debemos establecer el valor de las capas superpuestas como null:

```
L.control.layers(baseMaps, null, {collapsed:false}).addTo(map);
```

El resultado sería este:



Un aspecto interesante del control es la posibilidad de añadir un icono antes o después del nombre, basta con incluir un pequeño código HTML que incluya la etiqueta img y la fuente de la imagen:

```
var overlayMaps = {
       "<img
src='https://upload.wikimedia.org/wikipedia/commons/thumb/e/e4/Google_Earth_icon.svg/
240px-Google_Earth_icon.svg.png' height=15px, width=15px /> Paises": paises,
       "<img src='https://img.icons8.com/small/16/000000/city.png'/>Ciudades":
ciudades
};
```



Por defecto el control se colapsa en un icono y se expande al pasar el ratón por encima o hacer clic.

Para que el control de capas se muestre siempre desplegado podemos añadir la opción `collapsed` a la clase `L.control.layers`. De esta manera establecemos su valor a false: `{collapsed:false}`

Por lo que quedaría así:

```
L.control.layers(baseMaps, overlayMaps, {collapsed:false}).addTo(map);
```

Ejercicio nº7. Haz una captura de pantalla con el resultado del mapa `L_map7.html`

El código fuente utilizado es el siguiente:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-
scale=1.0, user-scalable=no" />
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

<style>
               body {
               padding: 0;
               margin: 0;
               }
               html, body, #map {
               height: 100%;
               }

</style>

 </head>
  <body>
   <div id="map"></div>
       <script>
```

```
            var osm =
            L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
                attribution: '&copy; <a
            href="http://osm.org/copyright">OpenStreetMap</a> contributors',
               maxZoom: 18
            });

            var paises = L.tileLayer.wms("http://localhost:8080/geoserver/wms", {
                layers: 'countries',
                format: 'image/png',
                transparent: true,
                attribution: "NaturalEarth"
            });

            var ciudades = L.tileLayer.wms("http://localhost:8080/geoserver/wms", {
                layers: 'cities',
                format: 'image/png',
                transparent: true,
                attribution: "NaturalEarth"
            });

      //Añadimos los dos objetos
      var baseMaps = {
          "OpenStreetMap": osm
      };

      var overlayMaps = {
            "<img
      src='https://upload.wikimedia.org/wikipedia/commons/thumb/e/e4/Google_Earth_icon
      .svg/240px-Google_Earth_icon.svg.png' height=15px, width=15px /> Paises":
      paises,
           "<img src='https://img.icons8.com/small/16/000000/city.png'/>Ciudades":
      ciudades
      };

      //Creamos el mapa
      var map = L.map('map', {
          center: new L.LatLng(41.66, -4.72),
          zoom: 4,
          //Añadimos las capas al mapa
          layers: [osm, paises, ciudades]
      });

      //creamos el control de capas y lo añadimos al mapa
      L.control.layers(baseMaps, overlayMaps, {collapsed:false}).addTo(map);

       </script>
 </body>
 </html>
```

Control de títulos y subtítulos

Haz una copia del archivo `L_map2.html` y renómbralo como `L_map8.html`. Añade este código al final:

```
// Insertando un título en el mapa

var title = L.control();
```

```
title.onAdd = function (map) {
        var div = L.DomUtil.create('div', 'info');
            div.innerHTML +=
            '<h2>Ciudades del mundo</h2>Tamaño según población';
         return div;
};

title.addTo(map);
```

Mediante este código, hemos creado un nuevo control denominado **`title`**, es necesario utilizar una función que cree un bloque div.

Este bloque div contiene un código HTML en el que incluimos el texto que irá como título h2 y como párrafo.

Por último, vamos al apartado del estilo para modificar el estilo de nuestro título mediante CSS:

```
<style>
  #map {
  width: 100%;
  height: 600px;
  box-shadow: 5px 5px 5px #888;
  }
  .info {
        padding: 6px 8px;
        font: 14px/16px Arial, Helvetica, sans-serif;
        background: white;
        background: rgba(255,255,255,0.8);
        box-shadow: 0 0 15px rgba(0,0,0,0.2);
        border-radius: 5px;
}
.info h2 {
        margin: 0 0 5px;
        color: #777;
}
</style>
```

Ejercicio nº8. Añade el título al mapa en el mapa L_map8.html y haz una captura de pantalla con el resultado.

Control de leyenda. Añadiendo una bonita leyenda al mapa

Haz una copia del archivo `L_map2.html` y renómbralo como `L_map9.html`. Añade el siguiente código (si te fijas el código es muy similar al anterior):

```
// Insertando una leyenda en el mapa

var legend = L.control({position: 'bottomright'});

legend.onAdd = function (map) {
        var div = L.DomUtil.create('div', 'info legend');
            div.innerHTML +=
            '<img src=" leyenda.png " alt="legend" width="101" height="60">';
        return div;
};

legend.addTo(map);
```

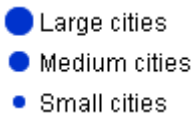
Ojo! El archivo *leyenda.png* no está incluido en los datos del curso, por lo que no aparecerá en el mapa.

En lugar de apuntar a un archivo PNG estático, podemos utilizar la respuesta del servicio WMS de GeoServer ***GetLegendGraphic***, que devuelve la leyenda de una capa (o grupo de capas) como un archivo PNG.

Por ejemplo, si introduces esta url en el navegador:

```
http://localhost:8080/geoserver/wms?REQUEST=GetLegendGraphic&VERSION=1.0.0&FORMAT=image/png&WIDTH=20&HEIGHT=20&LAYER=earth:cities
```

Aparecerá la leyenda que creamos en GeoServer con el complemento de estilos CSS:



La petición *GetLegendGraphic* utiliza el estilo asignado por defecto a la capa, si te aparece otra leyenda al visualizar la anterior url, tienes dos opciones:

a) Ir a GeoServer y asignar el estilo *cities_filter* a la capa ***cities***.
b) Utilizar esta otra url:

```
http://localhost:8080/geoserver/wms?REQUEST=GetLegendGraphic&VERSION=1.0.0&FORMAT=image/png&WIDTH=20&HEIGHT=20&LAYER=earth:cities&Style=cities_filter
```

El código final quedaría así:

```
// Insertando una leyenda en el mapa

var legend = L.control({position: 'bottomright'});

legend.onAdd = function (map) {

        var div = L.DomUtil.create('div', 'info legend');

            div.innerHTML +=
            '<img src="
http://localhost:8080/geoserver/wms?REQUEST=GetLegendGraphic&VERSION=1.0.0&FORMAT=ima
ge/png&WIDTH=20&HEIGHT=20&LAYER=earth:cities" alt="legend" width="101" height="60">';

        return div;

};

legend.addTo(map);
```

Por último, si se desea, se puede modificar el estilo de nuestra leyenda mediante CSS, para aplicar una transparencia a la leyenda:

```
.legend {
       opacity: 0.7;
}
```

Para mostrar varias capas WMS en la leyenda es necesario crear un grupo:

```
http://localhost:8080/geoserver/wms?REQUEST=GetLegendGraphic&VERSION=1.0.0&FORMAT=ima
ge/png&WIDTH=20&HEIGHT=20&LAYER=worldgroup
```



Ejercicio nº9. Añade la leyenda de la capa ***cities*** con el estilo ***cities_filter*** en el mapa L_map9.html y haz una captura de pantalla con el resultado.

Capas vectoriales. El formato GeoJSON

GeoJSON se está convirtiendo en un formato de datos muy popular entre muchas tecnologías y servicios GIS. **¿Por qué?** es simple, ligero, sencillo, y Leaflet es bastante bueno en su manejo.

Leaflet soporta todos los tipos geométricos de geoJSON. Pero los tipos "***Feature"*** y "***FeatureCollection"*** son los más adecuados para trabajar, ya que nos permiten describir los elementos con un conjunto de propiedades. Incluso podemos utilizar estas propiedades para dar estilo a nuestros vectores de Leaflet. He aquí un ejemplo de un *feature* GeoJSON simple:

```
var geojsonFeature = {
    "type": "Feature",
    "properties": {
        "name": "Reino de León",
        "amenity": "Campo de fútbol de la Cultural y Deportiva Leonesa",
        "popupContent": "Aquí es donde juega el equipo de fútbol de León"
    },
    "geometry": {
        "type": "Point",
        "coordinates": [-4.99, 39.75621]
    }
};
```

Trabajando con archivos GeoJSON en Leaflet

En este ejercicio crearemos un mapa de coropletas interactivo de los países del mundo.

Haz una copia del archivo `L_map1.html` y renómbralo como `L_map10.html`

Modifica el código y pon al comienzo el objeto map, sin la opción layers.

Cambia la escala de visualización a 4.

```
var map = L.map('map', {
              center: [41.66, -4.72],
              zoom: 4
       });
```

Visualizaremos el mapa con tiles de MapBox que tiene una bonita escala de grises y se ajustará bien con nuestra capa de países. Sustituye el código que mostraba los tiles de OSM, por este otro:

```
      {
    attribution: '© <a href="https://www.mapbox.com/feedback/">Mapbox</a> © <a
href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
    maxZoom: 18,
    id: 'mapbox/streets-v11',
    tileSize: 512,
    zoomOffset: -1,
   }).addTo(map);
```

Para que el código anterior funcione sin problemas hay que tener la precaución de comprobar que no hay saltos de línea incorrectos. La primera línea termina en {, y el resto terminan en una coma salvo la última que termina en ;

Además, como has observado Mapbox requiere de una clave, debes comprobar que está perfectamente escrita, incluyendo los guiones que contiene, en caso contrario no funcionará:

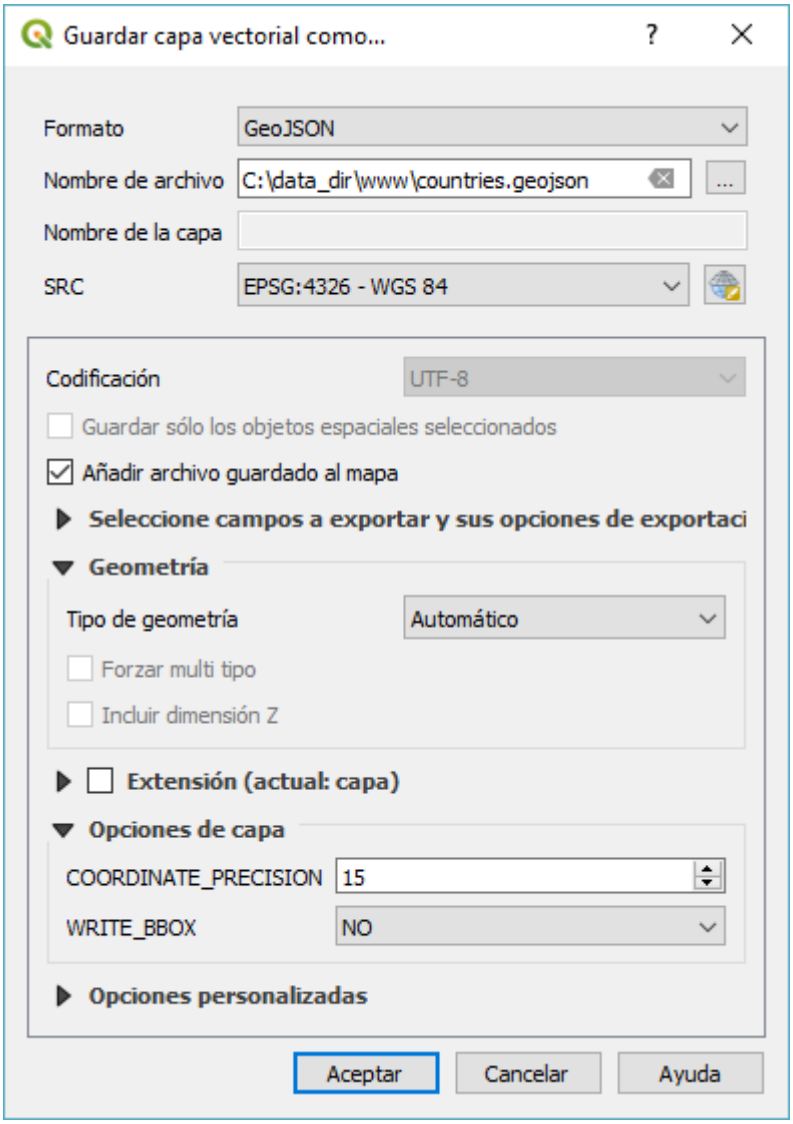
```
access_token=
```



Ahora vamos a trabajar con la capa en formato geoJSON. Utilizaremos la capa de países original en formato SHP *countries.shp.*

Ya hemos visto en ejercicios anteriores como convertir un **shapefile a geojson.** Tenemos varias opciones:

1. Utilizar la librería *open source* **ogr2ogr** que sirve para convertir datos vectoriales entre distintos formatos.
2. Utilizar un conversor online:
    a. HTML5 Shapefile to GeoJSON Converter. Usando HTML5 y es simple: arrastra tu shapefile y obtén un GeoJSON.
    b. MyGeodata Converter, que acepta hasta 20 formatos de entrada y 8 de salida (ESRI Shapefile, Microstation DGN, MapInfo File, CSV, GML, GPX, KML y GeoJSON).
3. Utilizar un cliente SIG como QGIS. Desde QGIS añadimos el SHP y a continuación guardaremos la capa en formato GeoJSON. (Capa > Guardar como...).



Guarda el nuevo archivo generado en el mismo directorio que las páginas HTML que estamos creando.

Cambia la extensión del archivo a "**countries.js**". Es importante cambiar la extensión de ***.geojson*** a ***.js*** para que funcione.

Ahora que hemos convertido el shapefile a JavaScript necesitamos asegurarnos de que Leaflet lo importará y reconocerá nuestros datos.

Con un editor de texto, como el Notepad++, abrimos el archivo countries.js y agregamos el texto

```
var countries =
```

Justo antes del código ya existente del archivo, de tal forma que el contenido del archivo countries.js sea:

```
var countries = {"type":"FeatureCollection", ... } // resto del archivo geojson
```

Haciendo esto, estamos estableciendo que el contenido completo del archivo GeoJSON sea igual a la variable `countries`.

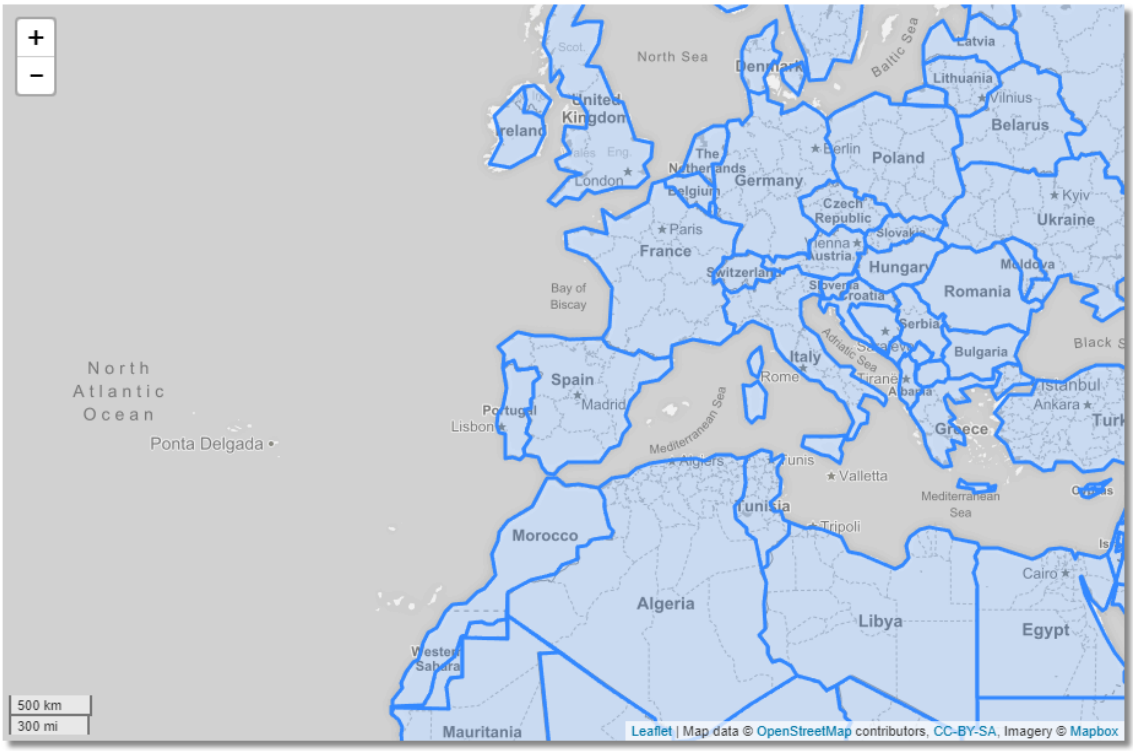
Una vez que hemos vinculado el archivo en el encabezado del documento del mismo modo que cualquier otro archivo JavaScript, ahora seremos capaces de manipular el archivo a través del objeto `countries`.

Lo llevaremos a nuestro documento, incorporando la siguiente sentencia a la cabecera:

```
<script type="text/javascript" src="countries.js"></script>
```

Los objetos GeoJSON se añaden al mapa a través de una capa de GeoJSON. Para crearla y añadirla un mapa, podemos usar el siguiente código:

```
L.geoJson(countries).addTo(map);
```



Un poco de color

Vamos a dar estilo a nuestra capa asignando los estilos en una variable:

```
var countryStyle = {
    'color': "#e12a2a",
    'weight':2,
    'opacity': 0.6
};
```

Como hemos visto en la presentación teórica, una capa vectorial de polígonos hereda las opciones de la clase **Path**.

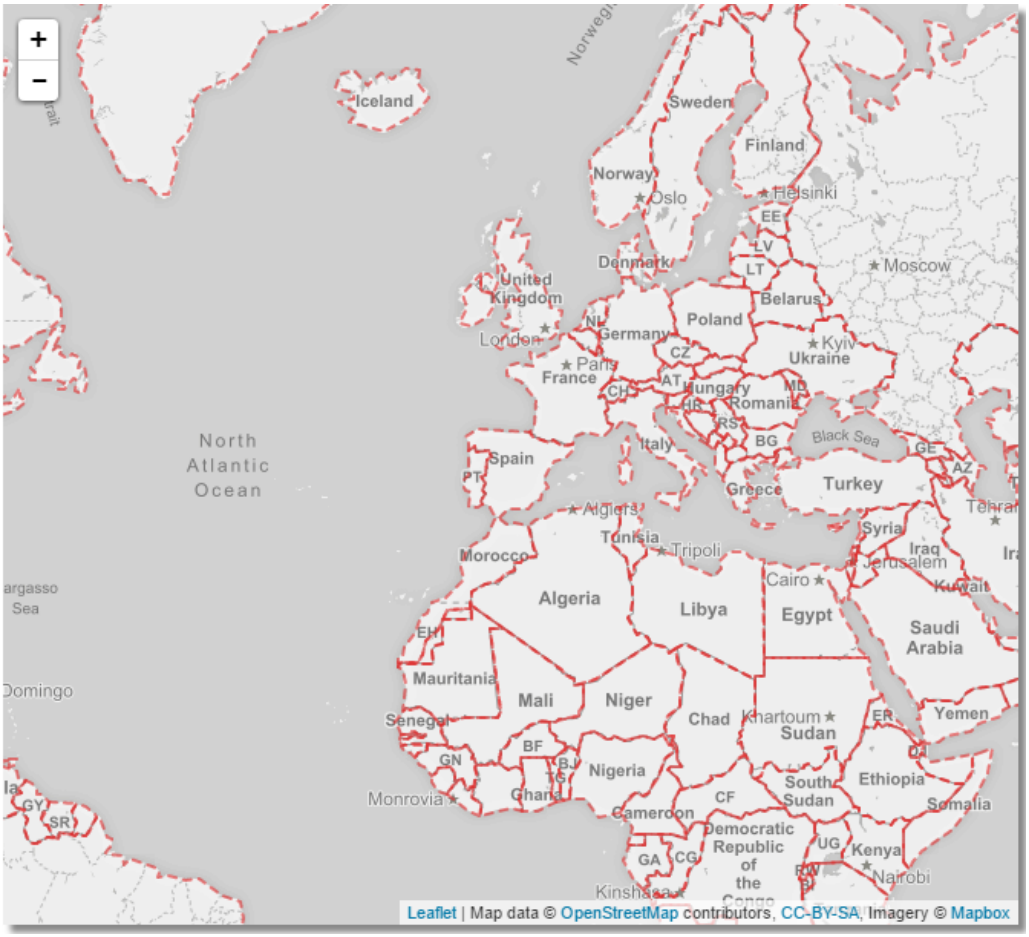
Las opciones de esta clase son:

`stroke, color, weight, opacity, fill, fillcolor, fillOpacity, dashArray, lineCap, lineJoin, clickable, pointerEvents, classname`

Juega con estas opciones para eliminar el color de relleno, poner un borde de tramas...

E incluimos el estilo nuevo a la capa GeoJSON:

```
L.geoJson(countries, {
       style: countryStyle
}).addTo(map);
```



Ahora vamos a dar color a los países de acuerdo a su población. La elección de colores agradables para un mapa puede ser una labor un poco complicada, pero hay una herramienta que puede ayudar, se trata de ColorBrewer.

Utilizando los valores que recibimos de él, creamos una función que nos devuelve un color en función de la población:

```
function getColor(d) {
    return d > 100000000 ? '#800026' :
           d > 50000000  ? '#BD0026' :
           d > 20000000  ? '#E31A1C' :
           d > 10000000  ? '#FC4E2A' :
           d > 5000000   ? '#FD8D3C' :
           d > 2000000   ? '#FEB24C' :
           d > 1000000   ? '#FED976' :
                           '#FFEDA0';
};
```

A continuación, definimos otra función para nuestra capa GeoJSON de manera que `fillColor` (el relleno de los polígonos) dependerá de la propiedad `feature.properties.pop_est`. Es decir, el campo población de nuestra capa de países.

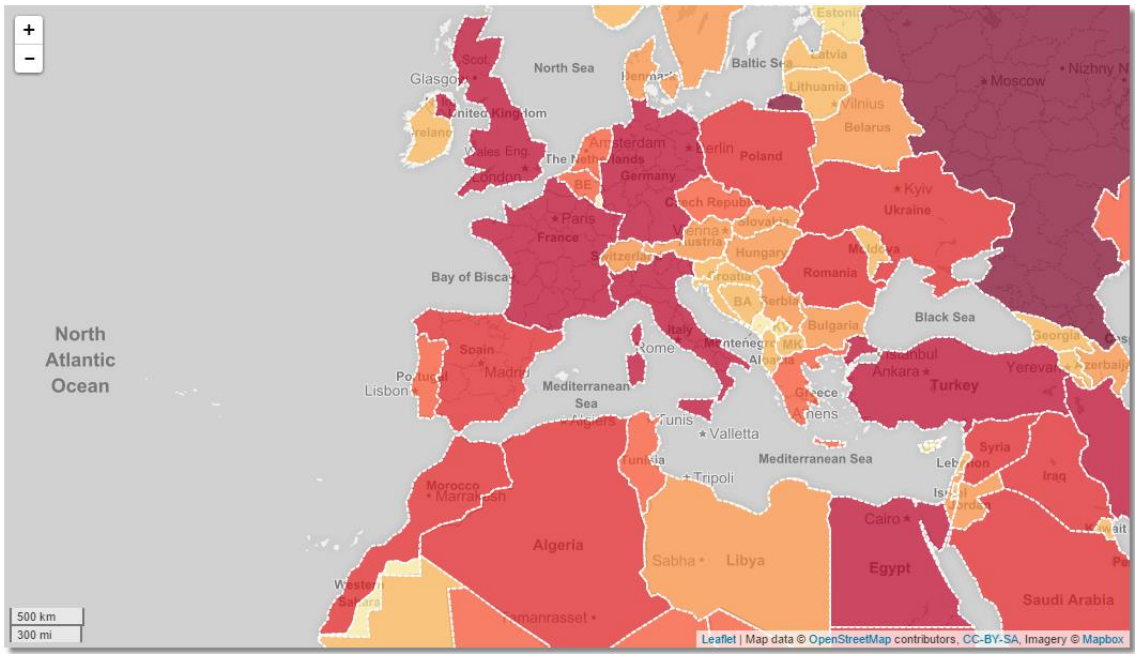
También ajustaremos la apariencia y añadimos un borde blanco con puntos discontinuos.

```
function style(feature) {
    return {
        fillColor: getColor(feature.properties.pop_est),
        weight: 2,
        opacity: 1,
        color: 'white',
        dashArray: '3',
        fillOpacity: 0.7
    };
}
```

Cambia el estilo para que se refleje el nuevo estilo:

```
L.geoJson(countries, {
        style: style
}).addTo(map);
```

Este sería el resultado:



Interactuando con el mapa

Ahora vamos a añadir una funcionalidad definiendo una función que entre en las propiedades de los datos de nuestro geojson. El siguiente código usa la funcionalidad `popup` de Leaflet:

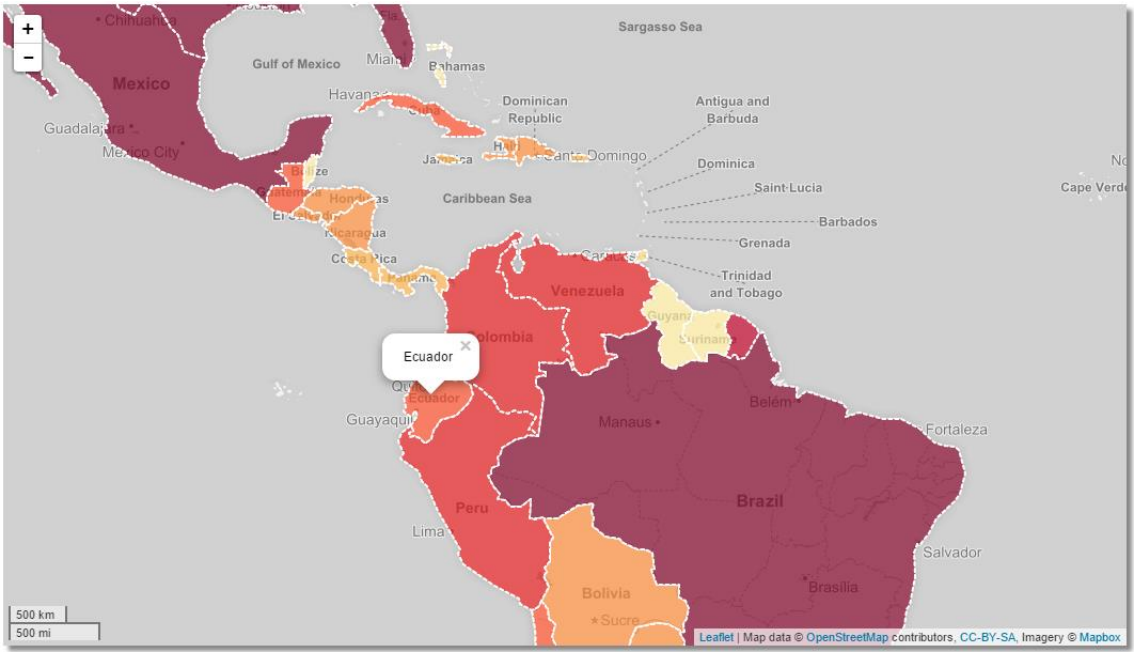
```
function popup(feature, layer) {
    if (feature.properties && feature.properties.name) {
        layer.bindPopup(feature.properties.name);
    }
}
```

Esta función comprueba que la parte ***properties*** del GeoJSON existe, así como la propiedad del campo ***name***, que es único para el archivo de datos.

La función `layer.bindPopup` construye la funcionalidad de clic en el mapa como hemos visto anteriormente.

`onEachFeature` es una propiedad de `L.geoJson` que se puede pasar con una función:

```
geojson = L.geoJson(countries, {
    style: style,
    onEachFeature: popup
}).addTo(map);
```



Ejercicio nº10. Haz una captura de pantalla con el resultado del mapa `L_map10.html`

Añadir una capa WFS de GeoServer y mostrar en una ventana emergente sus atributos mediante AJAX

En este ejercicio crearemos un mapa interactivo de los países del mundo utilizando la capa contries como servicio WFS

Haz una copia del archivo `L_map1.html` y renómbralo como `L_map11.html`

Modifica el código y pon al comienzo el objeto map, sin la opción layers.

Cambia la escala de visualización a 2.

```
var map = L.map('map', {
       center: [41.66, -4.72],
       zoom: 2
});
```

En primer lugar, añadiremos al script la librería jQuery:

```
<script src="https://code.jquery.com/jquery-3.5.1.js" ></script>
```

JQuery es una librería de JavaScript de código abierto, que simplifica la tarea de programar en JavaScript y permite agregar interactividad a un sitio web sin tener conocimientos del lenguaje.

Vamos a construir la URL con el servicio WFS de GeoServer donde debemos reanudar nuestro geoJSON.

La dirección URL servirá para crear la llamada AJAX a la página que contiene nuestro GeoJSON:

```
var owsrootUrl = 'http://localhost:8080/geoserver/ows';

var defaultParameters = {
       service: 'WFS',
       version: '1.0.0',
       request: 'GetFeature',
       typeName: 'earth:countries',
       outputFormat: 'application/json'
};

var parameters = L.Util.extend(defaultParameters);

var URL = owsrootUrl + L.Util.getParamString(parameters);
```

La llamada AJAX se construye mediante JQuery, a través de la cual se pasa la URL y una función de éxito, o qué efecto debe realizarse la llamada.

AJAX (JavaScript Asíncrono y XML) nos proporciona la posibilidad de hacer peticiones al servidor (intercambiar datos) sin tener que volver a cargar la página.

En nuestro ejemplo, vamos a hacer clic sobre los objetos geográficos de la capa *countries*, y mostraremos un popup que contendrá la información captada por GeoJSON. En ese caso, veremos el nombre del país.

El método **$.ajax ()** realiza una solicitud asíncrona AJAX:

```
$.ajax({
    url: URL,
    success: function (data) {
       //creamos una nueva capa geojson
        var geojson = new L.geoJson(data, {
             //Aplicamos un estilo
             style: {"color":"#2ECCFA","weight":2},
             //vinculamos un popup mostrando el nombre del país
             onEachFeature: function(feature, layer){
                  layer.bindPopup("Has hecho click en " + feature.properties.name);
             }}
       ).addTo(map); }
});
```



Ejercicio nº11. Haz una captura de pantalla con el resultado del mapa `L_map11.html`

Extendiendo la funcionalidad de Leaflet mediante plugins

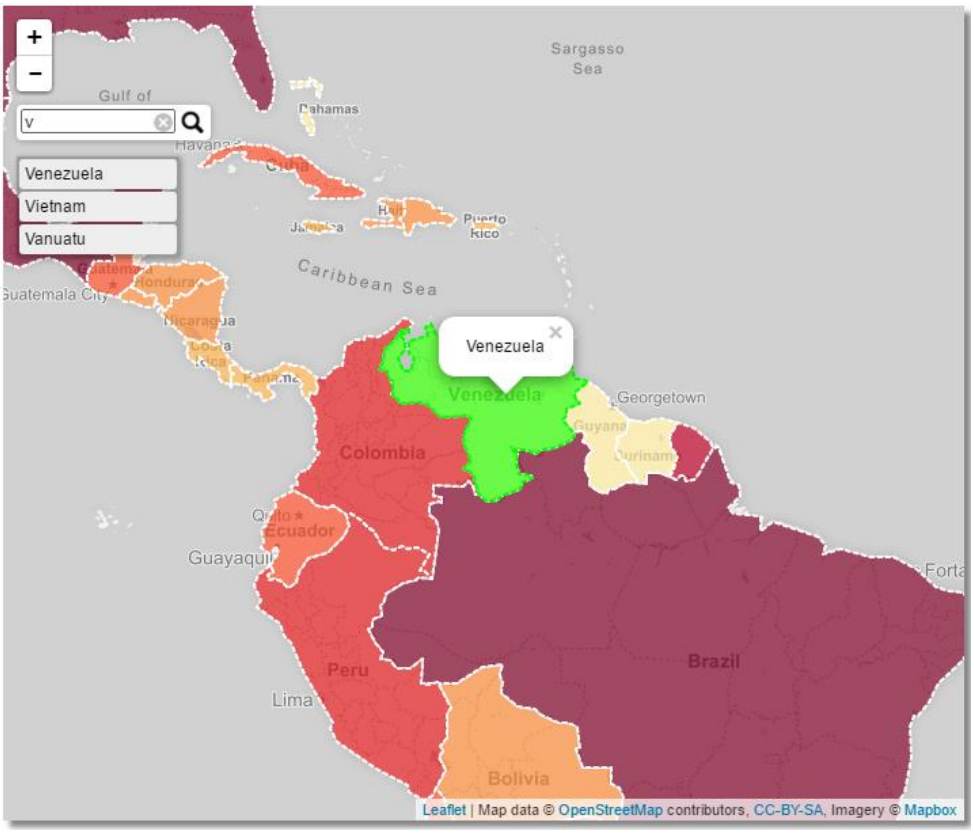
Sin duda, una de las principales ventajas de Leaflet es el gran número de plugins de que dispone. La librería de Leaflet es muy ligera, contiene los elementos imprescindibles para crear el mapa. Podemos completar este cuerpo básico con los plugins.

Insertando un control de búsqueda de atributos en una capa estática GeoJSON

Vamos a continuar dando interactividad a nuestro visor web incorporando **un control** que realice una búsqueda en un campo de los atributos de una capa en formato GeoJSON.

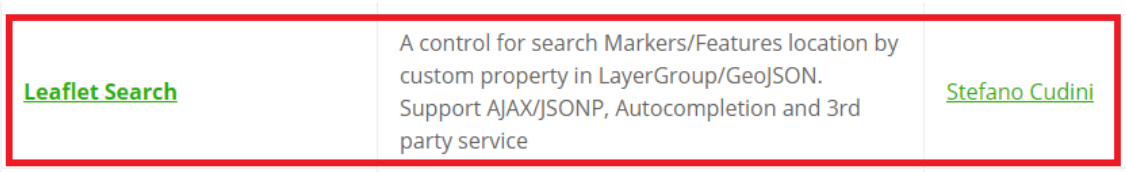
Añadiremos a nuestro visor web un control que realice una búsqueda mediante atributos dentro de los objetos geográficos vectoriales.

El control buscará en marcadores o en los objetos geográficos de un GeoJSON.

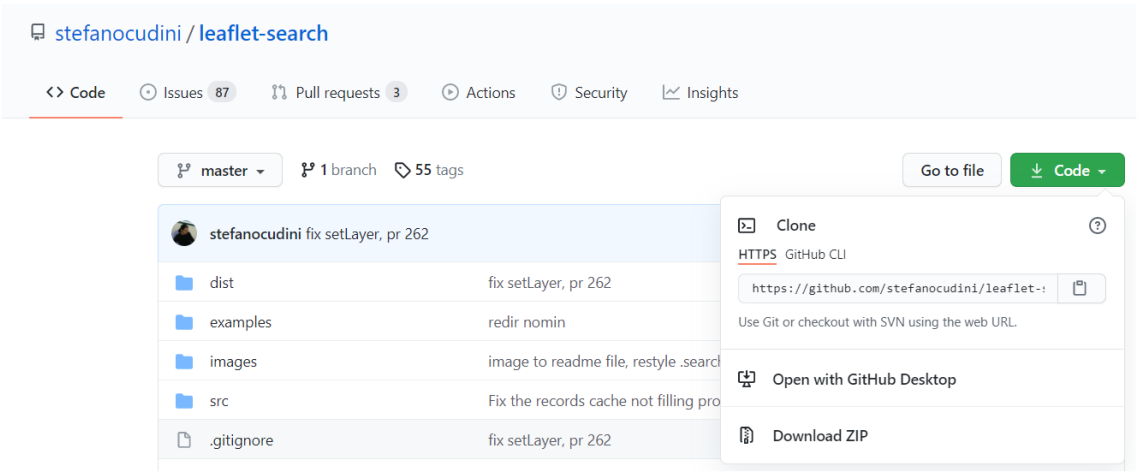


Haz una copia del archivo `L_map10.html` (que ya debe contener la capa *countries* en formato GeoJSON) y renómbralo como `L_map12.html`

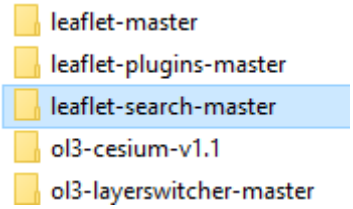
1. Ve a la página de plugins de Leaflet: http://leafletjs.com/plugins.html
2. Ve a la Sección: **Overlay interaction** > Search & popups y pincha sobre ***Leaflet Search***. Este es el plugin para añadir las capas de Google en Leaflet:



3. Hacemos clic sobre el menú Code. A continuación, nos descargamos el plugin haciendo clic sobre el Download ZIP, que se encuentra en la parte de abajo del menú.



4. Descomprimimos el archivo y **lo colocamos en el mismo directorio en el que se encuentre nuestro HTML.**



5. Debajo del script que añade la capa countries.js, añadiremos la librería JS y la hoja de estilos CSS en la cabecera del HTML:

```
<script src="leaflet-search-master\src\leaflet-search.js"></script>
<link rel="stylesheet" href="leaflet-search-master\src\leaflet-search.css" />
```

6. Creamos una variable denominada countriesJS para meter en ella la capa de países que tenemos en formato GeoJSON:

```
var countriesJS = L.geoJson(countries, {
        onEachFeature: popup,
        style: style
}).addTo(map);
```

7. A continuación creamos una nueva variable, denominada `searchControl`, que contendrá el control de búsqueda, que se implementa a través de la clase `L.Control.Search` :

```
var searchControl = new L.Control.Search({
        layer: countriesJS,
        propertyName: 'name'
});
```

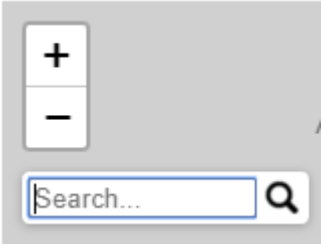
En el código superior vemos como las propiedades de la clase `L.Control.Search` son `layer` y `propertyName`, en las que incluiremos la capa sobre la que queremos realizar la búsqueda (`countriesJS`) y el campo que contendrá el texto a buscar (`name`) respectivamente.

8. Por último, añadimos el control al mapa:

```
map.addControl(searchControl);
```

Listo. Esta es la manera más simple de añadir el control de búsqueda de atributos en una capa GeoJSON.

Haz clic en el nuevo botón con forma de lupa e introduce el nombre de un país (en inglés):



Comprobarás que, al hacer clic sobre el resultado de la búsqueda, en el país aparece una circunferencia roja. Esto es útil si trabajamos con una capa de puntos, pero en una capa de polígonos, lo interesante es resaltar el polígono con otro color.

Esto sucede porque la opción de mostrar la localización mediante una circunferencia (`circleLocation`) viene por defecto como `true`.

Vamos a modificar el código para eliminar esta circunferencia `circleLocation: false`:

```
var searchControl = new L.Control.Search({
        layer: countriesJS,
        propertyName: 'name',
        marker: false
});
```

Inmediatamente después y antes de añadir el control al mapa, crearemos una función para definir el estilo del objeto geográfico resultado de la búsqueda:

```
searchControl.on('search:locationfound', function(e) {
        e.layer.setStyle({fillColor: '#3f0', color: '#0f0'});
})
```

Realiza de nuevo la búsqueda para comprobar el resultado.

El siguiente paso es limpiar los resultados después de cada búsqueda.

Inmediatamente después crearemos una nueva función que limpie los resultados de las búsquedas:

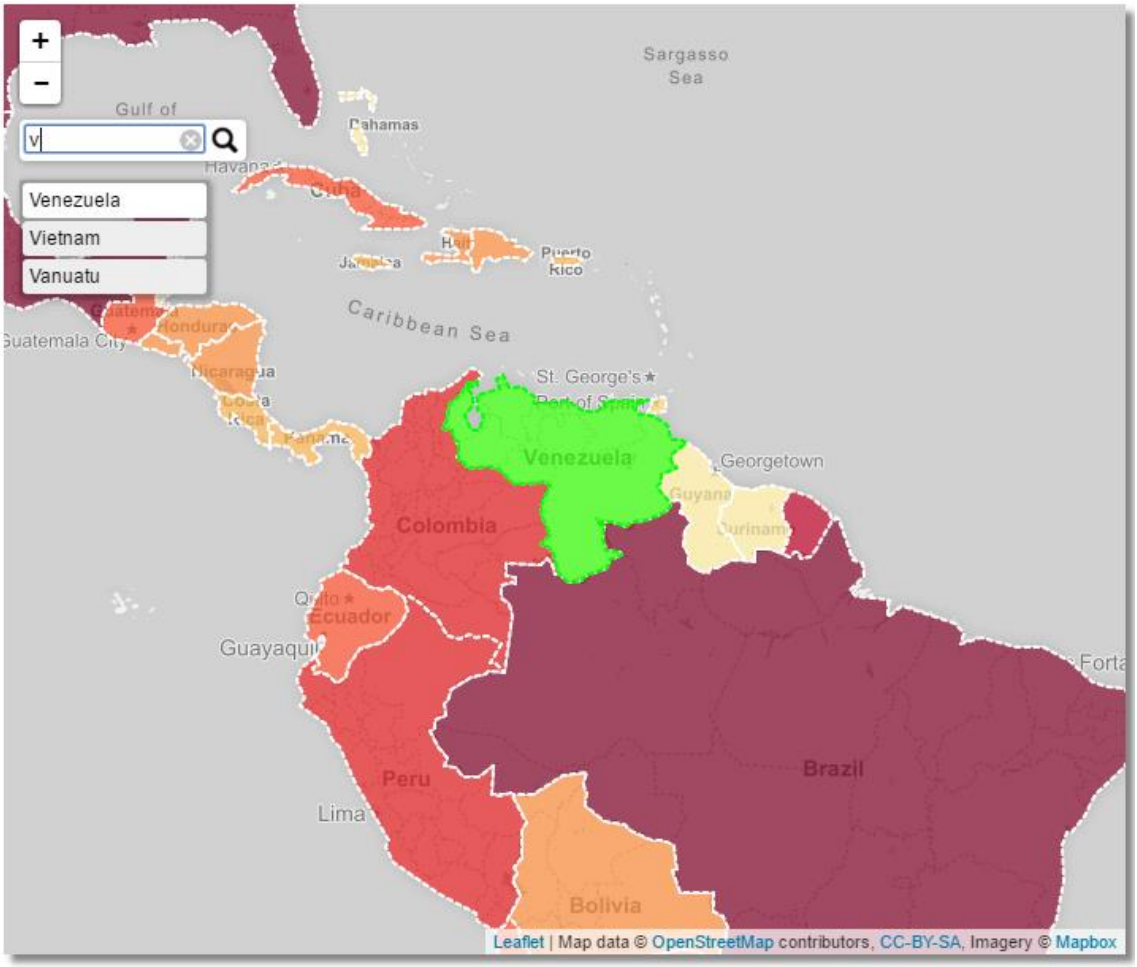
```
.on('search:collapsed', function(e) {
        countriesJS.eachLayer(function(layer) {    //restauramos el color del elemento
```

```
                countriesJS.resetStyle(layer)
        })
});
```

Y al final del script añadimos el control al mapa:

```
map.addControl(searchControl);
```

Listo. Al hacer clic sobre la ruleta del ratón se limpiarán los resultados.



Aquí tienes el HTML completo:

```
<!DOCTYPE html>
<html>
<meta charset="utf-8" />
<head>
   <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/>
   <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
   <script type="text/javascript" src="countries.js"></script>
<script src="https://cdn.jsdelivr.net/npm/leaflet-search@4.0.0/dist/leaflet-
search.src.min.js"></script>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/leaflet-
search@4.0.0/dist/leaflet-search.src.min.css">
 <style>
  #map {
         width: 100%;
         height: 600px;
         box-shadow: 5px 5px 5px #888;
  }
 </style>

 </head>
  <body>
   <div id="map"></div>
 <script>

       var map = L.map('map', {
              center: [41.66, -4.72],
              zoom: 4
       });

       L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(map);

       var countryStyle = {
              'color': "#858585",
              'weight': 2,
              'opacity': 0.6
       };

       function getColor(d) {
       return d > 100000000 ? '#800026' :
              d > 50000000 ? '#BD0026' :
              d > 20000000 ? '#E31A1C' :
              d > 10000000 ? '#FC4E2A' :
              d > 5000000 ? '#FD8D3C' :
              d > 2000000 ? '#FEB24C' :
              d > 1000000 ? '#FED976' :
                            '#FFEDA0';
       };

       function style(feature) {
              return {
                     fillColor: getColor(feature.properties.pop_est),
                     weight: 2,
                     opacity: 1,
                     color: 'white',
                     dashArray: '3',
                     fillOpacity: 0.7
              };
       }

       function popup(feature, layer) {
              if (feature.properties && feature.properties.name) {
```

```
                layer.bindPopup(feature.properties.name);
        }
    }

    var countriesJS = L.geoJson(countries, {
        onEachFeature: popup,
        style: style
    }).addTo(map);


    var searchControl = new L.Control.Search({
        layer: countriesJS,
        propertyName: 'name',
        marker: false
    });

    searchControl.on('search:locationfound', function(e) {
        e.layer.setStyle({fillColor: '#3f0', color: '#0f0'});
    })
    .on('search:collapsed', function(e) {
    countriesJS.eachLayer(function(layer) {//restauramos el color del elemento
                countriesJS.resetStyle(layer)
        })
    });

    map.addControl(searchControl);

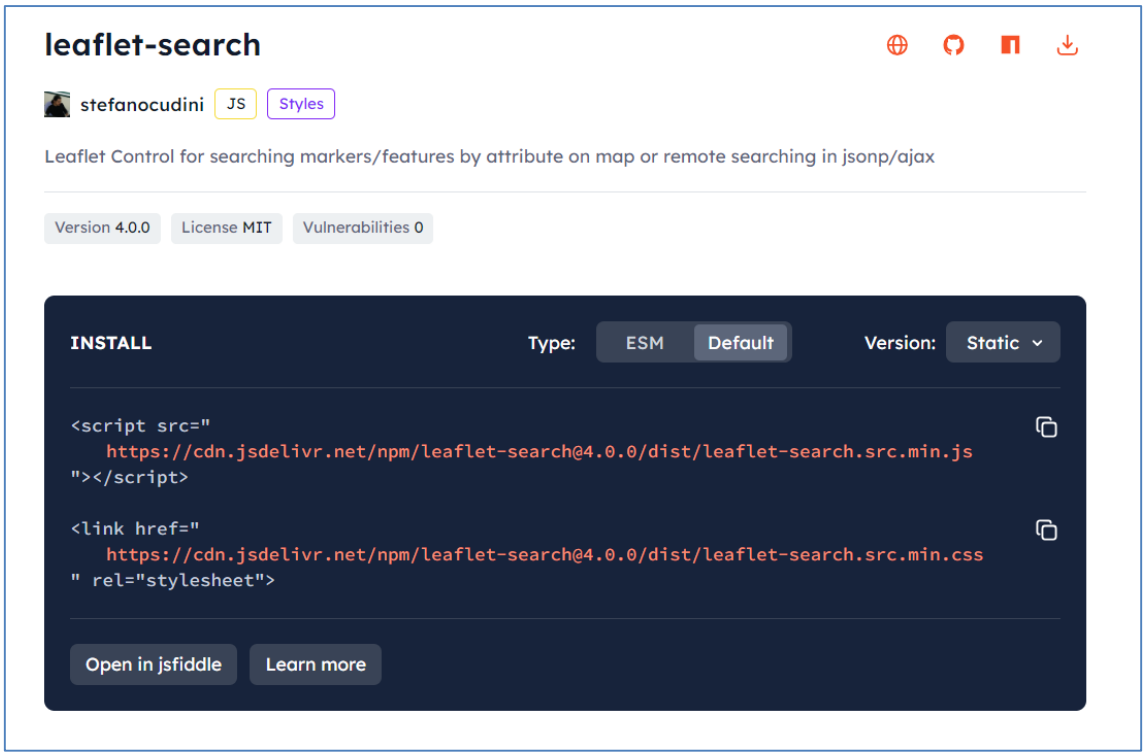
</script>
</body>
</html>
```

Como puedes observar en el código anterior, hemos sustituido la ruta a los archivos descargados del plugin por un enlace CDN. De esta forma puedes ver un ejemplo de las dos formas en que se puede utilizar un plugin de Leaflet:

1. Mediante la descarga de archivos.
2. Mediante un enlace CDN.

Hay varios sitios donde podemos encontrar los enlaces CDN con los plugins, en este caso hemos utilizado el siguiente:

https://www.jsdelivr.com/package/npm/leaflet-search



De este modo solamente debes copiar los enlaces que se indican y pegarlos en tu código.

Ejercicio nº12. Haz una captura de pantalla con el resultado del mapa `L_map12.html`